Who Sees What — Solution Architecture and Usage

Package: Who Sees What — User Access, Field Security & Record Visibility Explorer
Namespace: `twinforce` **Package Id:** `0HoOW00000000ZV0AY` **Version submitted:** `[FILL IN — e.g. 0.3.0-1 / 04t...]` **Publisher:** `[FILL IN — legal entity name]`
Technical contact: `[FILL IN — name and email]` **Date:** `[FILL IN]`

---

1. What the product does

Who Sees What is a read-only administrative tool. A System Administrator selects a user and the tool reports what that user can see and do — apps, objects, fields — and which permission set, permission set group, or profile granted each item.

It grants nothing, changes no permissions, and modifies no records other than its own audit log.

The problem it solves: in an org with dozens of permission sets and permission set groups, answering "why can this user see this field?" by hand means opening many Setup pages and mentally unioning the results. Muting permission sets make this harder still, because a naive union produces a confidently wrong answer.

---

2. Information flow

There are no external systems. All flow is inside the subscriber's org.

```
Admin's browser
  |  LWC: permissionExplorer + 21 child bundles
  |  Imperative Apex calls only. No fetch, no XHR.
  v
PermissionExplorerController   (@AuraEnabled entry points)
  |
  +-- PEAccessGate.assertAuthorised()   <-- ALWAYS FIRST
  |      FeatureManagement.checkPermission(
  |          'Use_Permission_Explorer')
  |      throws NOT_AUTHORISED if absent
  |
  +-- PEPermissionResolver    effective + muting parents
  +-- PEUserSearchService     user list, keyset paginated
  +-- PEAccessQueryService    object CRUD, field FLS
  +-- PEAppVisibilityService  app list (estimated)
  +-- PEUserDetailService     login history, drill-downs
  +-- PEMetadataCatalog       describe, labels, sharing
  +-- PECacheManager          cache + no-cache fallback
  +-- PEInspectionAuditLogger writes PE_Inspection_Log__c
  |
```

```
  v
Salesforce database -- all SOQL in user mode
```

Data touchpoints

Direction	What	Where
Read	Permission metadata and user records (15 sObjects, listed below)	Subscriber org
Write	One audit row per inspection	`PE_Inspection_Log__c`, subscriber org
Cache	Resolved permission results, describe metadata	Subscriber's Platform Cache
Outbound	**None**	—

Objects read

`User`, `UserLogin`, `UserRole`, `Profile`, `PermissionSet`, `PermissionSetAssignment`, `PermissionSetGroup`, `PermissionSetGroupComponent`, `PermissionSetTabSetting`, `ObjectPermissions`, `FieldPermissions`, `LoginHistory`, `AppMenuItem`, `TabDefinition`, `EntityDefinition`.

Every one of these is permission or user metadata. The package reads no business data — no Accounts, Contacts, Opportunities, or custom business objects. It reads the *permission rows describing* such objects, never the records.

---

3. Authentication and encryption on data transfer

Authentication: none performed by the package. It executes inside the subscriber's existing authenticated Salesforce session. No login flow, no credentials, no tokens, no Auth Providers, no Connected Apps.

Encryption on data transfer: not applicable — there is no data transfer off platform. Browser-to-server traffic is the standard Salesforce Lightning transport, encrypted in transit by the platform.

Callouts: zero. The package contains no `HttpRequest`, no `new Http()`, no `callout:` reference, no Named Credential, no External Credential, and no Remote Site Setting. Verifiable by searching the package source for those tokens.

This is a deliberate design decision with a product cost, described in §6.

---

4. Authorisation model

Three layers, all enforced by the platform rather than by our own checks.

Layer 1 — custom permission gate. Every `@AuraEnabled` method begins with `PEAccessGate.assertAuthorised()`. That method calls `FeatureManagement.checkPermission('Use_Permission_Explorer')` and throws a typed `NOT_AUTHORISED` exception if the running user lacks it. The decision is deliberately **not** cached in a static: an authorisation decision held in a mutable static outlives the context it was made in, and `checkPermission` is cheap enough that the guarantee is worth more than the microseconds.

Layer 2 — sharing. All 16 non-test classes are declared `public with sharing`.

Layer 3 — user-mode SOQL. Every query executes in user mode, so object- and field-level permissions are enforced by the platform for the running admin:

- Inline SOQL: `WITH USER_MODE`
- The one dynamic query: `Database.queryWithBinds(soql, binds, AccessLevel.USER_MODE)`

An admin therefore cannot use this tool to see permission metadata they are not themselves entitled to read.

Permission set contents. `PermissionCheckerPermissionSet` grants:

- The custom permission `Use_Permission_Explorer`
- Apex class access to `PermissionExplorerController` only
- `PE_Inspection_Log__c`: Read ✔️ Create ✔️ **Edit ✘ Delete ✘**, View All ✘, Modify All ✘
- Field permissions on the three non-required audit fields
- Tab visibility for `Permission_Explorer`

The withheld Edit and Delete are the point: a holder can read the audit log but cannot alter or erase their own trail.

The package does **not** auto-assign this permission set on install. It grants the ability to inspect every user's permissions, so assignment is left as a deliberate administrator decision. The in-app onboarding panel states this explicitly.

---

5. Auditing

Every inspection writes one row to `PE_Inspection_Log__c`:

Field	Meaning
`Inspector_User__c`	Who performed the inspection
`Subject_User__c`	Who was inspected
`Action__c`	`RESOLVE_USER`, `GET_APP_OBJECTS`, `GET_OBJECT_DETAIL`, etc.
`Inspected_At__c`	UTC timestamp

Field	Meaning
`Detail__c`	Context, e.g. the object API name inspected

Writes are buffered and flushed once per transaction. The object is append-only for permission set holders by design.

Known limitation, stated plainly: if the audit insert fails, the failure is logged with `System.debug` and the inspection still returns. In a managed package a debug statement is not visible to the subscriber, so an audit write failure is currently silent to them. We judged failing the admin's inspection over a logging error to be the worse outcome, but we accept this is a trade-off rather than an ideal, and we are willing to change the behaviour if the reviewer prefers a surfaced warning or a hard failure.

---

6. Deliberate limitations and how they are disclosed

The product's design principle is that it must never present a guess as an answer. Where the platform cannot tell us something without a callout, the interface says so in place rather than rendering an empty result that looks like "no access."

Area	Why not analysed	How disclosed
Per-user record type visibility	Stored only in Profile/PermissionSet metadata XML; no queryable junction object exists. Reachable only via Metadata or Tooling API — a callout.	Record types are listed as they exist on the object, under a persistent "Record type access is not analysed" notice naming `recordTypeVisibilities` as the source.
Per-permission-set app visibility	Same reason.	App list is derived from `AppMenuItem` and every row is labelled "visibility estimated," with a standing notice above the column.
Login history	Salesforce restricts `LoginHistory` to users with "Manage Users," which this package does not grant.	Where the running admin lacks it, the panel reports the reason instead of showing an empty list.
Protected permission sets	Permission sets installed by other managed packages may be unreadable. Their permissions still apply.	The header reports how many were skipped, so the admin knows the grid is a lower bound.
Sharing rules / record-level access	`__Share` queries can be enormous and OWD needs a callout to read fully.	Out of scope; the object detail panel shows the org-wide default only.

We consider these disclosures a feature rather than an apology: an admin who mistakes "not analysed" for "no access" would make a worse security decision than one who is told the boundary.

---

7. Scale and governor-limit handling

Keyset pagination, not OFFSET. SOQL `OFFSET` is capped at 2,000 rows, which silently truncates user lists in large orgs. `PEKeysetCursor` encodes an opaque cursor carrying the sort value and the last row Id; the Id tiebreaker is mandatory because the sort value is not unique, and without it rows duplicate or vanish across page boundaries. Asserted by `PEUserSearchServiceTest.pagingCollectsEveryRowExactlyOnce` and `lastLoginSortIsAcceptedAndPages`. Cursors are integrity-checked; a tampered cursor is rejected (`PermissionExplorerTest.tamperedCursorIsRejected`).

SOQL chunking. A user may have hundreds of effective permission-set parents, exceeding the bind-list ceiling for a single query. `PESoqlChunker` splits the parent set into batches and `assertQueryBudget` pre-checks the resulting query count against the governor limit before any query runs. This is why several methods query inside a loop — see the false-positives document.

Composite endpoint. `getObjectDetail` returns object permissions, field permissions, record types and sharing model in one round trip, so switching sections in the UI is pure client-side rendering.

Per-section error boundary. A field-list failure populates a typed error on the field section only; the object permissions panel that already rendered correctly is not blanked.

---

8. Usage instructions

Install and set up

1. Install the package (Admins Only recommended).
2. **Setup → Permission Sets → `PermissionCheckerPermissionSet` → Manage Assignments** — assign to the System Administrators who should be able to inspect other users. Not automatic, by design.
3. Check the App Launcher for **Who Sees What**. If absent: **Setup → Profiles** → your profile → **Assigned Apps → Edit** → enable **Who Sees What → Save**. (App assignment lives in profile metadata and cannot be granted by a permission set.)
4. Open **App Launcher → Who Sees What → Permission Explorer**.

Before a user is selected the tool shows an onboarding panel restating these steps, the capabilities, and the fact that use is logged.

Inspect a user

1. **User column** — search by name, username, alias or email, or filter by profile, role, or user type. Sort by name or last login. Results are keyset paginated.

2. **Select a user** — the header shows profile, role, licence, last sign-in, and counts of assigned permission sets and permission set groups. Each count opens a drill-down modal with links to the corresponding Setup page.
3. **App column** — apps visible to the user, each labelled "visibility estimated."
4. **Open an app** — objects and tabs reachable from it, with C/R/U/D/View All/Modify All chips and the granting source for each. Objects with data access but no tab in the app are surfaced separately, as a frequent cause of "why can they see this?" questions.
5. **Expand an object** — three tabs: **Fields** (readable/editable per field, with granting sources, and an explicit reason where FLS does not apply, such as formula or required fields), **Record types** (org-level list under the "not analysed" notice), **Origin** (where the object access came from).
6. **Copy link** shares the current selection; **Refresh data** bumps a cache version token rather than clearing the partition, so one admin refreshing does not cost every other admin their warm cache.

Verifying the audit trail

```
SELECT twinforce__Inspector_User__c, twinforce__Subject_User__c,
       twinforce__Action__c, twinforce__Inspected_At__c
FROM twinforce__PE_Inspection_Log__c
ORDER BY CreatedDate DESC
```

---

9. Test coverage

101 Apex tests, 100% passing, 88% package coverage.

Tests assert security properties, not only happy paths:

- `unauthorisedCallerIsRejectedByEveryEntryPoint` — the gate rejects on every entry point
- `unauthorisedUserIsRejectedBeforeAnyQuery` — rejection happens before any SOQL
- `quoteInTermIsBoundNotInterpolated` — search terms are bound, never interpolated
- `disallowedSortFieldIsRejected` — the sort allowlist holds
- `tamperedCursorIsRejected` — cursor integrity
- `pagingCollectsEveryRowExactlyOnce` — no duplicated or skipped rows across pages
- `mutingSubtractsAGrantedPermission`, `mutingDoesNotInventAGrant`, `mutingDoesNotLeakAcrossPermissions` — muting logic correctness
- `absentRowMeansNoAccessNotUnknown` — the absence rule

`PEUserDetailService` is capped near 57% by two platform constraints, documented in the class header: `Schema.isAccessible()` cannot be made to return false from a test because `System.runAs` enforces sharing but not CRUD, and `LoginHistory` rows cannot be inserted in a test context.

10. Static analysis

Salesforce Code Analyzer, AppExchange and Security rulesets: **0 violations.**

```
sf code-analyzer run --workspace force-app --config-file code-
analyzer.yml \
  --rule-selector AppExchange --rule-selector Recommended:Security
```

Six `@SuppressWarnings` annotations remain in `PEAccessQueryService`, none of which appear in the AppExchange or Security rulesets. Each is justified in the false-positives document and in the code itself.